CS6843 Program Analysis
Assignment 4 : Instrumenting Array Accesses

Deadline: November 2, 2025 Marks: 20

1 Problem Statement

The objective of this assignment is to write a transformation pass, to instrument array accesses with bounds check to ensure that out of bound read or write accesses are not performed. However, since these additional checks add performance overhead, we want to skip instrumenting the array accesses if the range analysis (from Assignment 2) computes the range of the index at the site to be within bounds, or identifies the site to be infeasible.

You will start with a template containing the range analysis computation from Assignment 2. You are expected to do the following in the same pass

- Update Range analysis part to handle any new instructions due to introduction of array accesses. Also update the range analysis to perform feasibility aware meet operation, i.e. when the range for any variable turns out to be empty set, then it implies that the path is infeasible, and so we need not merge such information.
- Instrument only the `test` function given in every testcase, which has the prototype `int test(int arg1, int arg2)`. In that function, iterate over every array accesses and instrument the access if (i) the range of the index variable is possibly out of bounds, (ii) range analysis does not indicate that the path is infeasible.
- For instrumentation, you are expected to instrument the GetElementPtr instruction (which is used for calculating the address into the array, before a load or store instruction). You are expected to do inline instrumentation, i.e. without calling any other functions for checking. When an out of bound access is detected, the control from must prevent from executing the GetElementPtr or the subsequent load/store instruction, and instead return `-1`.

1.1 Input

We will make the following simplifications to the input program:

- The program will only have local variables of type `int` (32-Bit Integers) and arrays of type `int`, and all of them will be declared at the beginning of the function. The function arguments will be integers (Arrays will not be passed across functions).
- There will not be any global variables, static variables, pointers, structures, unions in the program. The analysis can focus only on the integer local variables, and integer arrays.
- The program will only have addition, subtraction and multiplication operations on variables. The analysis can ignore other arithmetic operations.
- Conditions in loops and If-else control structures, will be of the form : `(var op k)` , where `var` will be a local variable, `op` represents one of $< \leq > \geq$ comparison operators and `k` is an integer constant.

- There will not be any nested loops in the program. (Every other control structure except nested loops have to be handled.)
- Each function will have a maximum of 30 lines of code.
- Arrays will be single dimensional integer arrays, declared as a local variable at the beginning of the function. The arrays will have an explicit size specified in the declaration.
- The arrays will be initialized with zeros. For the range analysis, treat the array as a single integer variable, merging writes the the array irrespective of the index.
- There will not be any pointer based accesses to the array. All array index in the program will only use another integer variable as index, not a constant value.

1.2 Output

For every testcase, for every array access that for which the instrumentation is skipped, print a line to `errs()` to indicate it as such. For instrumented array accesses, the output bitcode should contain the instrumentations, and on running the instrumented program, out of bound accesses should be prevented, and the program should return `-1` instead.

2 Example

Example program

```
1  #include<stdio.h>
2  #include<stdlib.h>
3
4  int test (int arg1, int arg2) {
5      int arr[20] = {0};
6      int i = 20, j = 20, k = 10;
7      if (i > 10) {
8          if (arg1 < 20) {
9              if (arg1 >= 0) {
10                 k = arr[arg1];  // Safe access, since arg1 range is [0,19]
11                 // So Not instrumented
12             }
13             k = arr[arg2];  // Instrumented here since arg2 range is [INT_MIN,
       INT_MAX]
14         }
15     } else {
16         // Infeasible path, indicated by range of i = [EMPTY]
17         // And so we won't instrument any array accesses in this branch
18         k = arr[j];     // Not instrumented
19     }
20     return 0;
21 }
22
23 int main(int argc, char **argv) {
24     printf("%d\n", test(atoi(argv[1]), atoi(argv[2]), atoi(argv[3])));
25     return 0;
26 }
```

Explanation:

In function `test`, we have defined an array `arr` of size `20`. For the array access in line `10`, due to the conditions in line `9` and `10`, we know that the index cannot go out of bounds, so instrumentation is skipped in this case. For the array access in line `13`, `arg2` is unconstrained and so we instrument this access, to ensure that it doesn't go out of bounds. For the array access in line `18`, because of the condition in line `7`, range of `i` in the else branch is empty, indicating that it won't get executed, so we don't instrument this access.

3 Template

The provided template contains the following files, folders and helper scripts:

- `analysis/arrayInstrumentation.cc` : The file containing the LLVM function pass template. You are expected to implement all your analysis logic in this file.
- `testcases/` : The testcases directory contains the public testcases. Each testcase consists of a C file, the corresponding LLVM IR file and the expected output from the analysis.
- `build.sh` : Helper script that compiles the analysis pass.
- `run.sh <inputfile.c/inputfile.ll>` : Helper script that compiles and runs the analysis on the given input file (in C or in LLVM IR format (.ll)).
- `run_testcases.sh` : Compiles the analysis and runs all the public testcases on the analysis. This script uses *lit* (LLVM Integrated Tester) for testing, and will install *lit* on the first run.

4 Submission Guidelines

- Submit only the `analysis/arrayInstrumentation.cc` file renamed as `ROLLNO.cc` on Moodle. Ensure that all your code is implemented in the same file.
- Evaluation will be based on test cases. Ensure that your code prints the results in the expected format and passes the public testcases provided.